

🔗 本章目标

理解 TCP/UDP 与客户端-服务端模型;掌握套接字编程全流程:创建、绑定、监听、接受、连接、收发;用 RAII 封装套接字资源,处理粘包、字节序、超时与错误;能够手写 HTTP 请求并理解常见网络库的选型。

📦 前置知识

熟悉 C 的套接字接口或至少理解文件描述符;掌握第5章 RAII 与第7章错误处理思路。本章代码基于 POSIX 套接字(Linux/macOS),Windows 需要额外调用 WSStartup,差异见 §8.2。

§8.1 网络模型速览	§8.2 套接字与 RAII 封装	§8.3 TCP 服务端
§8.4 TCP 客户端	§8.5 HTTP 与网络库	§8.6 字节序、超时与安全

§8.1 网络模型速览

网络编程只需要抓住两层:TCP 与 UDP 之上的应用层协议,和 TCP/IP 之下的传输层。TCP 是面向连接的可靠字节流:三次握手建立连接,数据按序到达,丢失自动重传,适合需要完整性的场景(HTTP、文件传输、聊天);UDP 是无连接的数据报,不保证到达与顺序,延迟低,适合实时场景(音视频、游戏、DNS)。C 程序员已经熟悉 socket 系列函数,这些在 C++ 里原样保留——C++ 不发明新的系统调用,而是用 RAII、异常与标准库把同样的事情做得更安全。

客户端-服务端模型是网络编程的基本骨架:服务端创建套接字、绑定端口、监听,然后循环接受连接;客户端创建套接字、连接服务端地址,然后双方收发数据。全程的数据都是字节流,协议(HTTP、自定义消息)负责把这些字节组织成有意义的信息。理解这个模型,网络编程的七成概念就通了:剩下的三成是细节——粘包、字节序、超时、并发,本章逐一处理。

还有两个基础概念要提前建立:端口与套接字。一台机器上可以同时运行许多网络程序,端口号(0 到 65535)用来区分它们,服务端 bind 的是端口,客户端 connect 的也是端口;套接字则是"网络连接的文件描述符",读写它就像读写文件——C 程序员把文件描述符的直觉搬过来,网络代码立刻亲切一半。端口号小于 1024 的通常需要管理员权限,调试用 8000 以上的高位端口即可。

§8.2 套接字与 RAII 封装

C 的套接字代码有两个顽疾:文件描述符必须每路径 close,错误全靠 errno 与返回值判断。C++ 的做法是照搬第5章的内存管理经验:把 fd 包进 RAII 类,析构自动关闭;错误检查封装成辅助函数,统一抛出异常。这样套接字与内存一样,"创建即管理,结束即释放",任何提前 return 或异常路径都不会泄漏 fd——fd 是稀缺资源,泄漏多了进程就再也打不开文件。

平台差异先说清:Windows 上套接字不是普通文件描述符,使用前必须调用 WSStartup 初始化 Winsock,关闭用 closesocket 而不是 close,链接需要 ws2_32 库;Linux/macOS 直接可用。工程上通常用一层薄的平台封装把差异吸收掉(本章代码以 POSIX 为准),或者干脆用跨平台网络库,见 §8.5。

C 的写法	C++ 的写法	说明
int fd = socket(...); 手工 close	SocketGuard guard(fd);	C 每路径记得关闭,C++ 析构保证关闭
if (fd < 0) { perror(...); return; }	fail("socket") 抛异常	错误信息统一携带 strerror
结构体逐字段填 sockaddr_in	同样的字段,辅以 RAII 与检查	底层接口一致,外层体验不同

```

#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>           // close
#include <arpa/inet.h>        // sockaddr_in / htons
#include <sys/socket.h>       // socket / bind / connect

class SocketGuard {          // RAII:构造持有,析构关闭
public:
    explicit SocketGuard(int fd) : fd(fd) {}
    ~SocketGuard() { if (fd >= 0) close(fd); }
    int get() const { return fd; }
    SocketGuard(const SocketGuard&) = delete;      // 不可复制
    SocketGuard& operator=(const SocketGuard&) = delete;
private:
    int fd;
};

void fail(const std::string& msg) {
    throw std::runtime_error(msg + ": " + strerror(errno));
}

int main() {
    try {
        int fd = socket(AF_INET, SOCK_STREAM, 0);
        if (fd < 0) fail("socket");
        SocketGuard guard(fd);                  // 之后任何异常,fd 都会被关闭
        std::cout << "套接字创建成功 fd=" << fd << "\n";
    } catch (const std::exception& e) {
        std::cerr << "错误: " << e.what() << "\n";
        return 1;
    }
    return 0;
}

```

§8.3 TCP 服务端

服务端四步曲:socket 创建套接字,bind 绑定本机地址与端口,listen 进入监听状态,accept 接受连接。accept 是阻塞调用:没有客户端连接时它一直等,来一个连接返回一个新的套接字——原来的监听套接字继续等下一个。sockaddr_in 是地址结构体,填法有讲究:sin_family 固定 AF_INET,sin_addr.s_addr 用 htonl(INADDR_ANY) 表示"监听本机所有网卡",sin_port 必须用 htons 把端口号转成网络字节序——这些转换函数的来历见 §8.6。

单线程服务端一次只能服务一个客户端,处理完才 accept 下一个。要并发服务多个客户端,经典路线是每连接一个线程(第9章),事件驱动路线是 select/poll/epoll。别急着上 epoll:先写对单线程与多线程版本,理解阻塞模型,再考虑高并发。工程上成熟的选择是直接使用网络库(§8.5),套接字的细枝末节交给库去处理。

```

#include <iostream>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

void echoServer(int port) {
    int fd = socket(AF_INET, SOCK_STREAM, 0);    // 1 创建套接字
    if (fd < 0) throw std::runtime_error("socket 失败");
    SocketGuard guard(fd);

    sockaddr_in addr;                          // 本机地址
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);    // 监听所有网卡
    addr.sin_port = htons((uint16_t)port);       // 端口转网络字节序

    if (bind(fd, (sockaddr*)&addr, sizeof(addr)) < 0) {    // 2 绑定
        throw std::runtime_error("bind 失败");
    }
    if (listen(fd, 8) < 0) {                                // 3 监听,排队上限 8
        throw std::runtime_error("listen 失败");
    }

    for (int round = 0; round < 3; ++round) {    // 服务 3 个客户端
        int cfd = accept(fd, nullptr, nullptr);    // 4 阻塞等连接
        if (cfd < 0) throw std::runtime_error("accept 失败");
        SocketGuard cguard(cfd);                  // 连接套接字同样 RAII
        std::cout << "已接入客户端 " << round + 1 << "\n";
        close(cfd);                                // 演示用:立即断开
    }
}

int main() {
    try {
        echoServer(8899);
    } catch (const std::exception& e) {
        std::cerr << "服务端错误: " << e.what() << "\n";
    }
    return 0;
}

```

§8.4 TCP 客户端与收发循环

客户端更简单:socket 创建套接字,connect 连接服务端,然后收发。地址结构体用 inet_pton 把 "127.0.0.1" 这类点分十进制字符串转成二进制地址,127.0.0.1 是回环地址,指本机自己,最适合本地调试。connect 是阻塞调用,服务端不在线会一直等,所以要在 connect 之前设置超时,或者用非阻塞模式加 select——工程上先学会设置超时这一招。

收发循环是本节的灵魂。TCP 是字节流,recv 一次返回的字节数"恰好等于你要的"是巧合,少收、多收都正常:send 也可能只发出去一部分。因此发送要写"发完为止"的循环,sendAll 函数把剩余字节反复 send;接收同样要"收满为止",这就是 recvMsg 循环的来历。粘包的概念也源于此:两次 send 的内容可能被一次 recv 收到,或者一次 send 的内容被两次

recv 拆开。解法是给每条消息加定长头部(如 4 字节长度前缀),接收方先收满头部,再按长度收满内容——这是应用层协议的通用设计,HTTP、WebSocket 都是这个思路。

C 的写法	C++ 的写法	说明
recv 一次,假设收满	recvMsg 循环收满	TCP 流语义,必须循环收发
消息边界靠约定与运气	4 字节长度前缀协议	定长头 + 内容,根治粘包
阻塞 connect 可能永远等待	SO_RCVTIMEO / SO_SNDTIMEO	超时兜底,避免挂死

清单 8-3 客户端:connect 与收发循环

```

#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

// 发送全部字节:send 可能只发出部分,循环补齐
bool sendAll(int fd, const std::string& data) {
    size_t off = 0;
    while (off < data.size()) {
        ssize_t n = send(fd, data.data() + off, data.size() - off, 0);
        if (n <= 0) return false;
        off += (size_t)n;
    }
    return true;
}

// 接收完整消息:先收 4 字节长度,再收内容(与发送方协议一致)
std::string recvMsg(int fd) {
    uint32_t len = 0;
    size_t off = 0;
    while (off < 4) { // 收满长度头
        ssize_t n = recv(fd, (char*)&len + off, 4 - off, 0);
        if (n <= 0) return "";
        off += (size_t)n;
    }
    len = ntohl(len); // 还原主机字节序
    if (len > (1 << 20)) return ""; // 防御:拒绝超大消息
    std::string buf(len, '\0');
    off = 0;
    while (off < len) { // 收满内容
        ssize_t n = recv(fd, &buf[off], len - off, 0);
        if (n <= 0) return "";
        off += (size_t)n;
    }
    return buf;
}

int main() {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return 1;
    SocketGuard guard(fd);

    sockaddr_in addr;
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(8899);
    if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) <= 0) return 1;

    if (connect(fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
        std::cerr << "连接失败,服务端是否启动?\n";
        return 1;
    }
    if (!sendAll(fd, "你好,网络")) return 1;
}

```

```

std::string resp = recvMsg(fd);
std::cout << "回应: " << resp << "\n";
return 0;
}

```

⚠ 易错点 1:假设 recv 一次收完

int n = recv(fd, buf, 100, 0); 之后直接处理 buf,这是经典错误:TCP 是字节流,这次可能只到了 30 字节。一切接收都要"循环收满期望长度",一切发送都要"循环发完",协议设计则保证接收方知道期望长度。

§8.5 HTTP 与网络库

HTTP 是应用层最常用的协议,它本身也是"文本行 + 空行 + 正文"的结构:请求行(GET /index.html HTTP/1.1)、若干头部行、空行、可选的请求体。手写 HTTP 客户端是理解协议的最好练习:用 connect 连上 80 端口,把请求文本原样发过去,再用 recv 循环把响应收回来。注意服务端返回后不会主动断开,必须靠 Content-Length 或读取到连接关闭来判断响应结束。手写的版本能做实验、能学原理,但生产环境请用库:curl 命令行与 libcurl、cpp-httpplib(单头文件,适合小型服务)、Boost.Asio(底层与异步)、POCO。选型口诀:个人工具与教学用 cpp-httpplib,企业异步高并发用 Boost.Asio 或 Asio 系库,HTTP 客户端直接调 libcurl。

无论用哪个库,HTTP 的基础语义都要烂熟:状态码(200 成功、404 未找到、500 服务端错误)、请求方法(GET 取数据、POST 提交数据)、头部字段(Content-Length 正文长度、Content-Type 内容类型、Connection 连接策略)。调试网络程序的第一工具是抓包与命令行:curl -v 看完整交互,netcat 模拟任意 TCP 对话,它们在 C 时代就在,C++ 时代依然是排查协议问题的利器。

C 的写法	C++ 的写法	说明
手写 socket + 拼请求文本	cpp-httpplib / libcurl 一行调用	库封装细节,专注业务
解析响应靠手工字符串处理	库提供响应对象与状态码	C++ 生态成熟,错误处理完备
每个平台重写一遍	库跨平台	Windows/Linux/macOS 行为一致

清单 8-4 手写 HTTP GET 客户端

```

#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>

// 请求:连 80 端口,发 HTTP 请求文本,收响应
std::string httpGet(const std::string& host, const std::string& path) {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return "";
    SocketGuard guard(fd);

    sockaddr_in addr;
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(80); // HTTP 默认端口
    if (inet_pton(AF_INET, host.c_str(), &addr.sin_addr) <= 0) return "";
    if (connect(fd, (sockaddr*)&addr, sizeof(addr)) < 0) return "";

    std::string req = "GET " + path + " HTTP/1.1\r\n"
        + "Host: " + host + "\r\n"
        + "Connection: close\r\n\r\n"; // 收完自动断开
    if (!sendAll(fd, req)) return "";

    std::string resp;
    char buf[4096];
    for (;;) {
        ssize_t n = recv(fd, buf, sizeof(buf), 0);
        if (n <= 0) break; // 读到连接关闭
        resp.append(buf, (size_t)n);
    }
    return resp;
}

int main() {
    std::string r = httpGet("127.0.0.1", "/index.html");
    if (!r.empty()) std::cout << "响应头前 200 字符:\n" << r.substr(0, 200);
    return 0;
}

```

§8.6 字节序、超时与安全

字节序问题:C 程序员都知道 x86 是小端,网络协议统一用大端(网络字节序)。htonl/htons 把主机整数转成网络字节序,ntohl/ntohs 反向转换。规矩只有一条:凡是写入网络的多字节整数,一律先 hton;凡是从网络读到的多字节整数,一律先 ntoh——忘一次,跨机器通信立刻乱码。这正是 §8.4 长度前缀要 htonl、接收端要 ntohl 的原因。

超时与安全是网络程序的生存线:所有阻塞调用都设置超时(setsockopt 的 SO_RCVTIMEO/SO_SNDTIMEO,或者 select/poll 的等待时间),避免连接挂死;从网络读到的每一条数据都当作不可信输入:长度前缀要上限检查(防超大分配),字符串要检查终止符,数值要检查范围——网络对端可能是任何人的代码,缓冲区溢出与整数溢出的攻击入口都在这里。C 的缓冲区管理(固定数组、手工边界)在网络上几乎必出漏洞,C++ 的 string/vector 自动管理长度,配合防御性检查,能把一类经典漏洞挡在门外。

```
#include <iostream>
#include <arpa/inet.h>

int main() {
    uint16_t port = 8899;
    uint16_t net = htons(port);          // 主机转网络:大端
    uint16_t back = ntohs(net);          // 网络转主机:还原
    std::cout << port << " -> " << net
               << " -> " << back << "\n";    // 8899 -> ... -> 8899
    // 教训:发送方 htons,接收方 ntohs,成对使用
    return 0;
}
```

⚠ 易错点 2:字节序漏转换

sin_port 直接填 8899、长度前缀直接写整数、读回长度不 ntohs——每一条都是跨机器必炸的雷。检查口诀:见到"写入网络的多字节整数"就找 htons,见到"从网络读回"就找 ntohs,一个都不能少。

⚠ 易错点 3:阻塞调用不设超时

accept、connect、recv、send 默认都是阻塞调用,对端宕机、网络断开时可能无限等待,整个线程挂死。所有可能长时间阻塞的调用都要配超时:setsockopt 的 SO_RCVTIMEO/SO_SNDTIMEO 是最简方案,select/poll 给等待时间更灵活。上线前的压测要专门模拟对端消失,确认超时逻辑真的生效。

网络编程速查表

概念	语法示例	要点
创建套接字	socket(AF_INET, SOCK_STREAM, 0)	SOCK_STREAM 是 TCP,SOCK_DGRAM 是 UDP
绑定端口	bind(fd, (sockaddr*)&addr, len)	服务端必须,端口冲突返回错误
监听	listen(fd, 8)	第二个参数是排队上限
接受连接	accept(fd, nullptr, nullptr)	阻塞等待,返回新套接字
连接服务端	connect(fd, (sockaddr*)&addr, len)	客户端必须,失败检查返回值
发送	send / sendAll 循环	send 可能发部分,必须循环
接收	recv / recvMsg 循环	按协议长度收满,防粘包
长度前缀	htonl(len) + 内容	应用层协议的标准解法
地址转换	inet_pton(AF_INET, "1.2.3.4", &addr)	文本地址转二进制
字节序	htons / ntohs	网络字节序,进出都转换
回环地址	"127.0.0.1"	本机调试首选
超时	setsockopt(SO_RCVTIMEO)	防阻塞挂死,必设
HTTP 客户端	libcurl / cpp-httplib	生产环境用库,不手写
Windows 差异	WSAStartup + closesocket	需 ws2_32 库,平台封装吸收

自测题

- Q 1. TCP 与 UDP 的区别是什么?各自的典型场景?
- Q 2. 服务端的完整流程是什么?accept 返回的套接字与监听套接字有何不同?
- Q 3. 什么是粘包?长度前缀协议是如何解决它的?
- Q 4. 为什么 sin_port 和长度前缀都要做字节序转换?
- Q 5. 网络数据为什么必须当作不可信输入处理?举两个防御例子。

✓ 参考答案

1. TCP 面向连接、可靠、按序字节流,适合 HTTP、文件传输、聊天;UDP 无连接、不保证到达与顺序,延迟低,适合音视频、游戏、DNS。需要完整性选 TCP,追求低延迟可容忍丢包选 UDP。
2. socket 创建、bind 绑定、listen 监听、accept 循环接客。监听套接字只负责等待与接受,accept 返回的连接套接字才真正承载收发;监听套接字继续等下一个连接。
3. TCP 是字节流,没有消息边界,多次发送的内容可能合并到达,一次发送的内容也可能被拆分。长度前缀协议在每条消息前写 4 字节长度,接收方先收满长度再收满内容,边界精确还原。
4. 网络协议统一使用大端(网络字节序),而 x86 等主机是小端。不转换,整数在另一台机器上解读出的值就是错的;所有进出网络的多字节整数都必须显式转换。
5. 对端可能是恶意代码或损坏的客户端。防御例子:长度前缀加上限检查防止超大内存分配;字符串字段检查终止符与长度防止溢出;数值字段检查范围防止整数溢出,一切解析都加异常处理。

实验:回声服务器与客户端

本实验实现一个完整的"长度前缀回声"系统:服务端接受客户端连接,收到消息原样返回并带上前缀;客户端发送消息并打印回应。代码分三部分,覆盖点已在注释中标明。先在终端 A 运行服务端,再在终端 B 运行客户端。

实验 8•第一部分:工具层(覆盖:RAII 封装、错误转异常、sendAll 循环、长度前缀)

```

#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#include <sys/socket.h>
#include <cstdint>

class SocketGuard { // 覆盖:RAII,任何路径都关闭 fd
public:
    explicit SocketGuard(int fd) : fd(fd) {}
    ~SocketGuard() { if (fd >= 0) close(fd); }
    int get() const { return fd; }
    SocketGuard(const SocketGuard&) = delete;
    SocketGuard& operator=(const SocketGuard&) = delete;
private:
    int fd;
};

void fail(const std::string& msg) { // 覆盖:错误统一转异常
    throw std::runtime_error(msg + ": " + strerror(errno));
}

// 覆盖:sendAll—send 可能发部分,循环到发完
bool sendAll(int fd, const std::string& data) {
    uint32_t len = htonl((uint32_t)data.size()); // 覆盖:字节序转换
    size_t off = 0;
    while (off < 4) {
        ssize_t n = send(fd, (const char*)&len + off, 4 - off, 0);
        if (n <= 0) return false;
        off += (size_t)n;
    }
    off = 0;
    while (off < data.size()) {
        ssize_t n = send(fd, data.data() + off, data.size() - off, 0);
        if (n <= 0) return false;
        off += (size_t)n;
    }
    return true;
}

```

实验 8•第二部分:服务端(覆盖:socket/bind/listen/accept、recv 循环、阻塞模型)

```

// 覆盖:recvMsg—先收 4 字节长度头,再按长度收内容
std::string recvMsg(int fd) {
    uint32_t len = 0;
    size_t off = 0;
    while (off < 4) {
        ssize_t n = recv(fd, (char*)&len + off, 4 - off, 0);
        if (n <= 0) return "";
        off += (size_t)n;
    }
    len = ntohl(len); // 覆盖:网络字节序还原
    if (len > (1 << 20)) return ""; // 覆盖:防御超大消息
    std::string buf(len, '\0');
    off = 0;
    while (off < len) {
        ssize_t n = recv(fd, &buf[off], len - off, 0);
        if (n <= 0) return "";
        off += (size_t)n;
    }
    return buf;
}

// 覆盖:服务端四步曲—bind 绑定、listen 监听、accept 循环
void runEchoServer(int port) {
    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) fail("socket");
    SocketGuard guard(fd);

    sockaddr_in addr;
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    addr.sin_port = htons((uint16_t)port);
    if (bind(fd, (sockaddr*)&addr, sizeof(addr)) < 0) fail("bind");
    if (listen(fd, 8) < 0) fail("listen");
    std::cout << "回声服务已在 " << port << " 端口就绪\n";

    for (;;) {
        int cfd = accept(fd, nullptr, nullptr); // 覆盖:阻塞等待连接
        if (cfd < 0) fail("accept");
        SocketGuard cguard(cfd);
        std::string msg = recvMsg(cfd);
        if (!msg.empty()) {
            sendAll(cfd, "回声: " + msg); // 原样返回
            std::cout << "收到并回应: " << msg << "\n";
        }
    }
}

```

实验 8•第三部分:客户端与 main(覆盖:connect、超时设置、双端组合)

```

int main(int argc, char** argv) {
    if (argc > 1 && std::string(argv[1]) == "-s") {
        try {
            runEchoServer(8899);                // 服务端模式
        } catch (const std::exception& e) {
            std::cerr << "服务端错误: " << e.what() << "\n";
        }
        return 0;
    }

    int fd = socket(AF_INET, SOCK_STREAM, 0);
    if (fd < 0) return 1;
    SocketGuard guard(fd);

    timeval tv{2, 0};                          // 覆盖:超时兜底 2 秒
    setsockopt(fd, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

    sockaddr_in addr;
    std::memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(8899);
    if (inet_pton(AF_INET, "127.0.0.1", &addr.sin_addr) <= 0) return 1;
    if (connect(fd, (sockaddr*)&addr, sizeof(addr)) < 0) {
        std::cerr << "连接失败:先以 -s 启动服务端\n";
        return 1;
    }

    std::string req = "你好,网络";
    if (!sendAll(fd, req)) return 1;             // 覆盖:发送循环
    std::string resp = recvMsg(fd);             // 覆盖:接收循环
    std::cout << "服务端回应: " << resp << "\n";
    return 0;
}

```

实验覆盖点核对:RAII 封装(SocketGuard 双端使用)、错误转异常(fail 与 try/catch)、sendAll 发送循环、recvMsg 接收循环、长度前缀协议(htonl/ntohl 成对)、服务端四步曲(socket/bind/listen/accept)、阻塞模型(accept 等待)、客户端 connect、超时设置(SO_RCVTIMEO)、回环地址(127.0.0.1)。扩展练习:把回声改成"统计每个单词出现次数"的协议;或启动两个客户端观察单线程服务端的串行行为,再引入线程(第9章)实现并发。

工程建议 1:协议先行,再写代码

任何网络程序先写协议文档:消息格式、长度前缀、字节序、错误码、超时值。两端按文档实现,联调时间从"猜对端行为"缩短为"核对文档"。协议里还要写清最大消息长度,这是所有防御性检查的依据。

工程建议 2:生产环境用网络库

手写套接字适合学习与教学,生产环境优先 cpp-httpilib、Boost.Asio 或 libcurl:线程模型、超时、TLS 加密、HTTP/2 都是成熟实现,自己造的轮子在安全与性能上很难追平。TLS(https)尤其不要手写,交给 OpenSSL 与库封装。

本章小结

网络编程的地基:TCP 可靠字节流与 UDP 无连接数据报;套接字四步曲与客户端两步走;RAII 封装与异常处理让资源安全;长度前缀协议解决粘包;字节序与超时是跨机器通信的纪律;网络数据一律不可信。下一章把服务端真正用起来:并发与异常处理。